

Collections

Altri tipi dati

In Python, **ci sono delle strutture dei dati più complesse delle variabili**, nelle variabili fino ad adesso abbiamo salvato solo un tipo di dati contenuti e per salvare due valori interi devo scrivere 2 variabili per salvare due valori.

Abbiamo visto i tipi semplici come i numeri interi, numeri con la virgola, stringhe, booleani e i Numeri Complessi (vedi le proprietà .real e .imag); tuttavia Python ha anche diversi tipi di composti incorporati, i quali agiscono come dei container per gli altri types.

Questi tipi di composti sono:

Type name	Esempio	Descrizione
Liste	[1, 2, 3]	Collection ordinate
Tuple	(1, 2, 3)	Raccolta ordinata immutabile
<u>Dictionaries</u>	{'key1': 'valore1', 'key2': 2, 'key3': 3}	Mappatura non ordinata (chiave, valore)
Set	{1, 2, 3}	Raccolta non ordinata di valori univoci

e rientrano nella grande famiglia delle collections

Nelle collection possiamo salvare più valori nella stessa struttura.

Se noi vogliamo tenere traccia di tutti i valori assegnati, devo usare la collection:

le collection ci permettono di memorizzare tanti tipi di dati nella stessa struttura.

Le liste

Sono una sequenza ordinata di elementi di valori, questi valori possono essere tutti dello stesso tipo o di tipo diversi, tutti i valori che si trovano nella lista si chiamano elementi della lista.

Caratteristiche delle Liste

1. Ordinate:

le liste tengono gli elementi nell'ordine in cui sono stati inseriti

2. Mutabili:

Posso modificare il contenuto di una lista

3. Consentono i duplicati:

Nelle liste posso avere elementi con lo stesso valore



Python

```
1 list1=[1,2,3,4,5]
2 list2=['A','B','C']
3 list3=[7,8,9,"D","S","O"]
4 print(list1)
5 print(list2)
6 print(list3)
```

Nei primi due casi abbiamo due liste che contengono tutti elementi dello stesso tipo mentre nel terzo sono elementi di tipo diversi tra loro.

Può capitare che all'interno della lista, l'elemento è una lista anch'esso



Python

```
1 list4=[[2,3,4], 5,6]
```

In questo caso si dice nested list(liste annidate):

gli elementi `[2,3,4]` è una lista dentro la list4.

Per stampare gli elementi di una lista



Python

```
1 #display all list
2 list1=[1,2,3,4]
3 print(list1)
```

Se voglio stampare gli elementi della lista senza parentesi scriverò



Python

```
1 print(*list1)
```

Se voglio separare gli elementi della lista tra loro con un carattere preciso:



Python

```
1 print(*list1), # sep=*,/,|, etc.
```

La cosa importante è **sequenza ordinata di elementi**:

io posso accedere agli elementi o singolo elemento attraverso degli indici, questa cosa non è possibile tramite una struttura non ordinata.



Python

```
1 indexing
2 mylist= ["Alice", "Bob", "Charlie"]
```

Avendo una lista con N elementi, devo ricordare, come per le stringhe, che **l'indicizzazione di una lista è zero-based (parte da zero)**:

- Alice è posizione 0
- Bob posizione 1

In analogia con le stringhe noi possiamo accedere a un determinato elemento della lista.

Es:



Python

```
1 myList= ["Alice", "Bob", "Charlie"]
2 print(myList[0])
```

Io sto chiedendo l'elemento alla posizione 0 della mia lista.

Le liste possono sia avere indici positivi che negativi, perché si possono leggere in 2 modi diversi:

1. da sinistra verso destra (classico):
quindi il primo elemento ha l'indice 0, il secondo l'indice 1 e così via
2. da destra verso sinistra:
il primo valore diventa l'ultimo e l'ultimo valore diventa il primo.
Utilizzeremo quindi indici negativi



Python

```
1 myList=["Alice", "Bob", "Charlie"]
2 index= 0      1      2
3 reverse
4 index= -3     -2     -1
```



Python

```
1 print(mylist[-1])
2 >>> Charlie
```

 [L'operatore in:](#) ✓

Per individuare un determinato elemento in una lista usare l'operatore `in`.



Python

```
1 frutti = ["mela", "banana", "ciliegia"]
2 #Controllare se un elemento è presente nella lista
3 if "banana" in frutti:
4     print("La banana è nella lista!")
5 else:
6     print("La banana non è nella lista.")
7 #Controllare se un elemento NON è nella lista
8 if "pera" not in frutti:
9     print("La pera non è nella lista.")
```



Python

```
1 frutti = ["mela", "banana", "ciliegia"]
2 #Controllare se un elemento è presente nella lista
3 if "banana" in frutti:
4     print("La banana è nella lista!")
5 else:
6     print("La banana non è nella lista.")
7 #Controllare se un elemento NON è nella lista
8 if "pera" not in frutti:
9     print("La pera non è nella lista.")
```

Es:



Python

```
1 myList=["Andrea", "Benedetta", "Camilla",]
2 print(myList)
3 myList[2]="Carlo"
```

```
4 print(myList)
```

In questo caso gli ho detto di sostituire "Camilla" con "Carlo"

Posso farlo anche così:



Python

```
1 myList=["Andrea", "Benedetta", "Camilla",]  
2 print(myList)  
3 myList[-1]="Carolina"  
4 print(myList)
```

In questo esempio richiamando la lista (`myList`) gli ho detto di prendere l'indice `[-1]` (cioè l'ultimo valore della lista se la leggo da sinistra verso destro ma in questo caso, poiché sto usando un indice negativo, è il primo elemento all'interno della lista) e, poiché gli ho assegnato il valore stringa "Carolina" me lo sostituirà con questo.

Come prima l'output verrà visualizzato come



Python

```
1 Andre, Benedetta, Carolina
```

Posso accedere ai miei elementi solo tramite indice.

La lista può essere modificata con elementi nuovi o sostituirli

Come detto prima in una lista posso avere elementi duplicati:



Python

```
1 [2, 5, 2]  
2 ["x", "x", 3]
```

Utile nel caso in cui in mi tornano elementi con lo stesso valore

Funzione `.append(element)`

Supponiamo di volere aggiungere l'elemento Davide



Python

```
1 myList=['Andrea', 'Benedetta', 'Camilla']  
2 print(myList)
```

```
3 myList[-1]='Carolina'
4 print(myList)
5 myList.append('Davide')
6 print(myList)
```

Quindi la funzione `.append` aggiunge un elemento a in fondo alla lista .

Se ad esempio voglio aggiungere `Erald` alla lista:



Python

```
1 myList=["Andrea", "Benedetta", "Camilla"]
2 print(myList)
3 myList[-1]="Carolina"
4 print(myList)
5 myList.append("Davide")
6 print(myList)
7 myList.append("Erald")
8 print(myList)
```

L'elemento aggiunto sarà sempre l'ultimo della lista

Le funzioni di una lista

Funzione `.insert`

vuole due valori in input:

1. l'indice
2. l'elemento che si vuole inserire a quell'indice specificato.

Es:



Python

```
1 myList.insert(1, 'Andrea_2.0')
2 print(myList)
3 >>>['Andrea', 'Andrea_2.0', ...]
```

In questo caso aggiungo all'indice 1 `Andrea_2.0` .

Adesso invece voglio rimuovere `Andrea_2.0` quindi farò

Funzione `.remove()` :

toglie l'input che si vuole rimuovere.

Es:



Python

```
1 myList = ['Andrea', 'Andrea 2.0', 'Carolina', 'Erald']
2 myList.remove("Andrea 2.0") # Usa lo spazio corretto
3 print(myList)
```

Funzione `pop()`

- **rimuove l'elemento in una determinata posizione (indice)** .
basta dargli come argomento un determinato indice (sia negativo che positivo).



Python

```
1 myList.pop(3)
2 print(myList)
```

La funzione `Extend` :

Ci consente di estendere la lista con più valori insieme



Python

```
1 myList = ["Alice", "Bob", "Charlie"]
2 myList.extend = ["Robert", "Karl", "Kate"]
3 print(myList)
```

Funzione `.del`

serve per eliminare la lista, la memoria del mio computer che era stata usata per salvare la lista viene liberata e la lista all'interno del codice non esisterà più

Modo di scorrere delle liste (Slicing)

Dove si scrive gli indici per accedere agli elementi si scrive invece

```
print(myList[0:2])
```

- **il primo indice è quello di partenza**

- **l'ultimo è quello di arrivo**

Quindi significa che parto dal primo elemento al secondo mi stamperà solo l'indice 0 e l'indice 1, escludendo l'indice 2 .

Se cambia la posizione del nostro indice



Python

```
1 print (myList[1:1])
2
3 print(myList[:1])
4
```

Nel primo caso, lo slicing viene effettuato con un indice di partenza e un indice di arrivo entrambi uguali, ovvero 1:1 .

In Python, quando usi una sintassi di slicing con gli indici di partenza e di arrivo uguali, il risultato sarà una lista vuota, perché non c'è alcun elemento tra l'indice di partenza e quello di arrivo.

Nell'ultimo caso sto utilizzando un slicing che parte dall'inizio della lista (poiché il primo indice è omesso) e si estende fino all'indice 1, **escluso**. Quindi, l'elemento che viene selezionato è quello all'indice 0 (il primo elemento della lista).

Ovviamente posso fare lo slicing anche con gli indici negativi:



Python

```
1 print(myList[-2:]) #viene visualizzato gli ultimi due elementi della lista
```

“Recording 20241217135846.webm” non trovato.

I set

Sono una collezione non ordinata di elementi unici , che non hanno un ordine fisso e prevedibile. Possono essere pensati come i diagrammi di Venn

Caratteristiche dei set

1. Unicità:

ogni elemento di un set è unico , il che significa che **i duplicati vengono automaticamente rimossi.**

2. Non ordinato:

l'ordine degli elementi in un set non è garantito, quindi non è possibile accedere agli elementi tramite un indice (a differenza delle liste).

3. mutabili:

i set sono mutabili, quindi puoi aggiungere o rimuovere elementi da un set dopo la sua creazione.

cioè posso modificare i valori all'interno levandoli e aggiungendoli (come le liste).



Python

```
1 m: set = {"ciao", 1, 1, 1, 1}
2
3 print(m)
4 >>> {ciao, 1}
```

Quando stampi o osservi gli elementi di un set, **l'ordine potrebbe sembrare casuale**, ma non lo è realmente.

Python utilizza una struttura chiamata **hash table** per gestire i set.

Gli elementi sono archiviati in base al loro hash, che è una funzione che calcola un valore numerico unico per ciascun elemento.

Hash e memoria:

- **La posizione degli elementi in un set dipende dall'hash e dall'implementazione interna di Python, che varia anche a seconda della memoria disponibile o della versione di Python.**
- **Non dipende né dall'ordine di inserimento né dall'ordine alfabetico/lessicografico.**

☰ Esempio pratico dei duplicati nei set



Python

```
1 s = {"b", "a", "c"}
2 print(s) # Output: {'b', 'c', 'a'}
```

Se stampiamo questa cosa, ogni volta che la stampiamo l'ordine può variare

Nei set sono presenti delle ripetizione degli elementi, queste ripetizioni vengono cancellate:



Python

```
1 m: set = {"ciao", 1, 1, 1, 1, 0, 0, 0, "ciao"}
2
3 print(m)
4 >>> {0, 1, 'ciao'}
```

Questo torna utile se devi fare il parsing dei siti web.

Hash

Tornando al concetto di **hasher**:

Un hasher è una funzione che calcola un valore numerico (hash) per un oggetto.

Gli hasher sono dei numeri che vengono generati, sono una funzione che prende in input un elemento e restituisce un numero, le liste non sono "hashable" perché sono un tipo di strutture dati mutabili(cioè i loro contenuti possono essere modificati).

Di conseguenza, la lista non è "hashable" perché, essendo il suo valore mutabile, renderebbe l'hash incoerente e ciò creerebbe dei problemi in strutture dati come ad esempio i dizionari, che fanno affidamento sugli hash per cercare velocemente i valori.

Per gestire situazioni in cui è necessario utilizzare oggetti mutabili come le liste in contesti che richiedono l'uso di hash (ad esempio come chiavi in un dizionario o in un set), possiamo utilizzare un **wrapper**:

è una classe che incapsula l'oggetto mutabile (come la lista) e fornisce una versione immutabile dell'oggetto, che è hashable.

Sostanzialmente coprono la lista e così si può calcolare il tipo di hasher.



Python

```
1 m: set = {"ciao", 1, 1, 1, 1, 0, 0, 0, "ciao", [0]}
2
3 print(m)
4 >>> TypeError: unhashable type: 'list'
```

In questo esempio, difatti, vediamo come inserire una lista ([0]) all'interno di un set ci dia errore proprio per il principio che gli oggetti all'interno di un set devono essere hashable (cioè immutabili e avere un valore di hash che non cambi durante il ciclo di vita dell'oggetto).

Siccome le liste invece sono mutabili non possono essere utilizzate come elementi di un set.

☰ Per Ricapitolare

- Un **hasher** è una funzione che calcola un valore numerico (hash) per un oggetto.
- Gli oggetti **mutabili** come le liste non sono **hashable** perché il loro contenuto può cambiare, rendendo l'hash incoerente.
- Per utilizzare oggetti mutabili come liste in contesti che richiedono hash, è necessario utilizzare un **wrapper** che trasforma la lista in un tipo immutabile (come una tupla), che può essere hashato.

🔗 Tip & Tricks su VS Code

per cambiare, ad esempio il nome di una variabile ed ho più occorrenze con quel nome con `doppio click` sul nome del variabile e poi `ctrl + d` e posso cliccarlo per tutte le occorrenze presenti

L'indexing dei set

Proviamo a fare l'indexing dei set:



Python

```
1 p: set = {"ciao", 1, 1, 1, 1, 0, 0, 0, "ciao"}
2 print(p[0])
3 >>> p[0]
4      ~^^^
5 TypeError: 'set' object is not subscriptable
```

come si vede mi da errore perché i set non hanno l'indexing poiché i set non sono ordinati.

Questo perché come abbiamo detto prima un set è una collezione non ordinata di elementi unici che non hanno un ordine fisso e prevedibile, ciò significa che non si può accedere a un elemento di un set utilizzando un indice.

Quindi fare `print[p0]`, è un errore perché non c'è nessun primo elemento in un set, visto che il loro ordine potrebbe cambiare ogni volta che accedi al set.

Funzioni dei set

1. la funzione `.add()` :

aggiunge un singolo elemento a un set.

Ovviamente se l'elemento da aggiungere è già presente nel set, il set non cambierà poiché i set non consentono i duplicati.

≡ Esempio pratico ▾



Python

```
1 s = {1, 2, 3}
2 s.add(4) # Aggiunge 4 al set
3 print(s) # Output: {1, 2, 3, 4}
4
5 #s.add(3) # Non aggiunge nulla perché 3 è già presente
6 #print(s) # Output: {1, 2, 3, 4}
7
```

2. La funzione `remove` :

rimuove un singolo elemento specifico dal set.

Se l'elemento non è presente, solleva un errore `keyError` .

≡ Esempio Pratico ▾



Python

```
1 s = {1, 2, 3, 4}
2 s.remove(3) # Rimuove 3 dal set
3 print(s) # Output: {1, 2, 4}
4
5 # Se provi a rimuovere un elemento che non >esiste, viene
  sollevato un errore
6 # s.remove(5) # Solleva un KeyError
7
8
```

3. La funzione `.union (o | )`:

==prendo il primo set e l'unisce al secondo set, senza i duplicati ==

≡ Esempio pratico: ▾



Python

```
1 x = {1, 2, 3}
2 y = {3, 4, 5}
3 z = x.union(y) # Unisce x e y
4 print(z) # Output: {1, 2, 3, 4, 5}
5
6 # Usando l'operatore |
7 z = x | y
8 print(z) # Output: {1, 2, 3, 4, 5}
9
```

4. La funzione `.intersection (o & )`:

==prendo il primo insieme e l'unisco al secondo, e mi restituisce un nuovo set che contiene solo gli elementi comuni tra i due set. ==

≡ Esempio Pratico ▾



Python

```
1 x = {1, 2, 3}
2 y = {3, 4, 5}
3 z = x.intersection(y) # Restituisce gli elementi comuni
4 print(z) # Output: {3}
5
6 # Usando l'operatore &
7 z = x & y
8 print(z) # Output: {3}
9
```

5. La funzione `.difference (o - )`:

La funzione `.difference()` restituisce un nuovo set che contiene gli elementi che sono presenti nel primo set, ma non nel secondo.

☰ Esempio Pratico ▾



Python

```
1 x = {1, 2, 3}
2 y = {3, 4, 5}
3 z = x.difference(y) # Restituisce gli elementi di x che non
  sono in y
4 print(z) # Output: {1, 2}
5
6 # Usando l'operatore -
7 z = x - y
8 print(z) # Output: {1, 2}
```

Quindi la sottrazione tra $x - y = a, c$ e mi leva z, k perché non gli interessano.

Se facessi l'operazione al contrario il risultato sarebbe $y - x = z, k$, quindi otterrei gli elementi di y che non sono presenti in x .



Python

```
1 z = y.difference(x) # Restituisce gli elementi di y che non
  sono in x
2 print(z) # Output: {4, 5}
3
4 # Usando l'operatore -
5 z = y - x
6 print(z) # Output: {4, 5}
```

I dictionaries

Sono una collezione non ordinata di coppie chiave-valore.

Ogni elemento del dizionario è costituito da una chiave unica (un oggetto immutabile) e un valore che può essere di qualsiasi tipo di dato.

I dizionari sono molto utili per associare informazioni in modo rapido e per accedere ai dati tramite chiavi.

(i file json sono un formato particolare che può essere letto in python e viene organizzato nei dictionaries e servono per la configurazione dove settiamo i valori dei parametri.

I json hanno delle chiavi-valori)

Caratteristiche principali

1. **Non ordinato** (fino a Python 3.6). A partire da Python 3.7, i dizionari mantengono l'ordine di inserimento, ma non è consigliato fare affidamento su di esso.
2. **Mutabile:**
puoi modificare, aggiungere o rimuovere coppie chiave-valore.
3. **Chiavi uniche:**
ogni chiave in un dizionario deve essere unica. Se provi a usare una chiave già esistente, il valore associato a quella chiave verrà aggiornato.
4. **Accesso rapido:**
l'accesso ai valori tramite le chiavi è molto veloce, grazie alla struttura dati sottostante.



Python

```
1 mydict = {key1:value1, key:value2, key3:value}
```

Non è possibile avere chiavi duplicate ma i valori invece si.

La sintassi

Un dizionario viene creato utilizzando il tipo `:dict` e con le parentesi graffe `{ }`, con ciascuna coppia chiave-valore separata da due punti `( : )`, e le coppie separate da una virgola.

Annidare i dizionari

Posso annidare altri dizionari dentro i dizionari.



Python

```
1 m:dict = {'key1':{'key1':"valore1"}}
2
```

```
3 print(m)
```

il più interno è il dizionario annidato è quello più esterno è il dizionario principale. Questo dizionario l'abbiamo chiamato `m`: contiene la chiave 1 e contiene un valore che è un altro dizionario che a sua volta contiene una chiave (key1) e che contiene un valore (valore 1).

Annidare cose dentro altre cose è una cosa comune, ma come inserire i valori e accedervi?

1. Per accedere ad un valore:



Python

```
1 m:dict = {"key1":5}
2 #per accedere al valore all'interno del dizionario
3 m = {'key1':5}
4 print(m['key1'])
5
```

così facendo accedo al valore del dizionario, ma se mi sbaglio ed inserisco una chiave che non esiste mi dà errore:



Python

```
1 m["key"]
2 traceback (most recent call last):
3 file "<stdin>", line 1, in <module>
4 key error: key
```

Come aggiungere cose all'interno inserendo chiavi che non esistono?

nel modo ; `m[key1]` **posso aggiungere cose nel dizionario** ad esempio:



Python

```
1 m: dict {} #dizionario vuoto, se la stampo ci apparirà {}
2 m["key"] = 1 # aggiungo un valore alla chiave "key"
```

Questo è solo un secondo metodo per inserire valori dentro al dizionario, ma questo modo si differenzia dal primo perché posso aggiungere valori e chiavi anche dopo la dichiarazione:



Python

```
1 m:dict={"key1": 1}
2 m["key"] = [1,2,3]
```

Inoltre si possono inserire delle liste all'interno del dizionario .

La ricerca tramite hash e per trovare valori all'interno del dizionario in maniera rapida.

Devo trovare dei valori dentro una lista disordinata, come si fa?

Da un punto di vista astratto per trovare un valore, devo scorrere tutti gli elementi.

Es:



Python

```
1 [1,3,2,5,7] #devo trovare il valore dieci e per farlo devo
   scorrere tutti gli elementi della lista
```

Se ho un miliardo di valori in una lista la verifica che devo fare è direttamente proporzionale ai valori presenti nelle lista:

Se ho un miliardo di elementi nella lista e io ho il valore interessato alla posizione 500 il computer scorrerà fino dalla posizione 1 alla posizione 500 per trovare quell'elemento.

Questo è un esempio di **ricerca lineare**:

Ricerca Lineare ($O(n)$)

immaginiamo di avere una lista disordinata:



Python

```
1 [1, 3, 2, 5, 7]
```

Per cercare un valore (ad esempio il numero 10), il metodo più semplice è scorrere ogni elemento della lista, confrontandolo con il numero che stiamo cercando.

Nella ricerca lineare dobbiamo fare un confronto per ogni elemento della lista. Se la lista ha n elementi, nel **caso peggiore** dovremo fare n confronti, ovvero il numero di operazioni è proporzionale alla lunghezza della lista. In notazione **asintotica**, questo si esprime come $O(n)$, dove n è la dimensione della lista. Ecco come funziona, passo dopo passo:

1. **Confronta il primo elemento della lista con il numero che stai cercando.**
2. **Se non è uguale, passa al secondo elemento e ripeti il confronto.**

3. Continua fino a trovare il numero o finire la lista.

Nel caso di una lista molto lunga, ad esempio con un miliardo di elementi, la ricerca lineare richiede un numero di operazioni direttamente proporzionale al numero di elementi. Per trovare un valore in una lista di $n = 1.000.000.000$ (un miliardo), nel **peggiore dei casi**, dovresti fare un numero di confronti pari a 1 miliardo di operazioni.

(Per fare ciò usa la notazione asintotica ($O(n)$): questo è un caso molto semplice per trovare un valore all'interno dell'algoritmo, cioè nel caso peggiore farò n operazioni per trovare il valore.).

Invece assumendo che la lista sopra sia ordinata per ordine crescente: come faccio per verificare che è numero all'interno della lista?

Se ad esempio cerco 10 escludo i numeri minori di 10, si chiama **ricerca binaria**.

Ricerca binaria $O(\log n)$

Ora, se la lista è **ordinata**, possiamo usare un altro algoritmo molto più efficiente chiamato **ricerca binaria**. Invece di scorrere la lista elemento per elemento, **la ricerca binaria divide la lista in due metà e si concentra solo sulla metà che potrebbe contenere il valore cercato.**

Per cercare il numero che ho all'ingresso (ad es: 10) e lo confronto con il centro della lista e mettiamo che al centro ci sia 3; io allora posso escludere la prima metà della lista e andrò a cercare nella seconda metà finché non trovo il numero 10.

Vediamo come funziona:

1. **Se la lista è ordinata, iniziamo confrontando il valore che stiamo cercando con l'elemento centrale della lista.**
2. **Se il valore che cerchiamo è maggiore del valore centrale, possiamo escludere tutta la metà inferiore della lista (tutti gli elementi a sinistra del centrale) e concentrarci solo sulla metà superiore.**
3. **Se il valore che cerchiamo è minore del valore centrale, possiamo escludere tutta la metà superiore della lista (tutti gli elementi a destra del centrale) e concentrarci solo sulla metà inferiore.**
4. **Ripetiamo il processo, riducendo sempre la lista a metà, fino a trovare il valore cercato o a ridurre la lista a zero elementi.**

Quindi, ad esempio, considerata la lista ordinata.



Python

```
1 [1, 2, 3, 5, 7]
```

Se stai cercando **10**, inizierai con l'elemento centrale:

- L'elemento centrale è **3**.
- Poiché 10 è maggiore di 3, escludi tutti gli elementi a sinistra di 3 e cerca solo nella metà destra della lista: `[5, 7]`.
- Ora prendi di nuovo l'elemento centrale, che è **5**.
- Poiché 10 è maggiore di 5, escludi tutti gli elementi a sinistra di 5 e cerca solo nell'elemento rimanente: `[7]`.
- Ora prendi l'elemento centrale, che è **7**.
- Poiché 10 è maggiore di 7, escludi tutto e concludi che 10 non è presente nella lista.

In questo caso, con la ricerca binaria, abbiamo ridotto la dimensione della lista da 5 a 1 in solo 3 passi!

Notazione Asintotica

La differenza tra ricerca lineare e ricerca binaria si riflette nella complessità computazionale, che indica quanto cresce il tempo di esecuzione in relazione alla dimensione della lista. Quindi la notazione asintotica è uno strumento matematico che ci permette di confrontare la complessità computazionale degli algoritmi:

1. Ricerca Lineare ($O(n)$):

Per una lista di n elementi, nel peggiore dei casi, dovrai eseguire n confronti.

La complessità è lineare.

2. Ricerca Binaria ($O(\log n)$):

Invece di fare un numero di operazioni proporzionale a n , la ricerca binaria riduce il problema a metà in ogni passo.

Quindi, se la lista ha n elementi, la ricerca binaria richiede solo circa $\log_2(n)$ passi, dove $\log_2$ è il logaritmo in base 2. Questo significa che, anche se la lista è molto lunga, il numero di operazioni cresce molto lentamente.

Esempio con 1 miliardo di elementi

Ritornando all'esempio con un miliardo di elementi, vediamo come le due ricerche si comportano:

- **Ricerca Lineare:**

Dovresti fare fino a un miliardo di confronti, quindi il numero di operazioni è $O(n)$.

- **Ricerca Binaria:**

Dovresti fare al massimo $\log_2(1.000.000.000)$ confronti. Il logaritmo di 1 miliardo in base 2 è circa 30, quindi ti servirebbero solo 30 operazioni per trovare il valore, anche se la lista è molto lunga.

Logaritmo

La funzione $O(n)$ (O di n) è una funzione lineare(sul grafico è una retta), mentre il $\log_2$ cresce lentamente perché assume più valori lentamente.

definizione di logaritmo:

è l'esponente che dobbiamo dare alla base per ottenere l'argomento .

```
1  log2 n = x
```

```
1  log2(8) = 3, perché  $2^3 = 8$ .
```

```
2  log2(16) = 4, perché  $2^4 = 16$ .
```

Quindi è l'esponente che dobbiamo dare a 2 per ottenere l'argomento.

Se noi quindi dobbiamo fare la ricerca su una lista e dobbiamo fare n operazioni e se ho una lista lunga un miliardo devo fare il $\log_2$ di un miliardo che è 30, quindi devo fare 30 operazioni.

In questo caso si usa il log in base due per nel caso della ricerca binaria divido la lista in 2 e poi ancora 2 e poi ancora 2 e così via.

☰ Per Ricapitolare

- **Ricerca Lineare ($O(n)$):**

Scorri tutta la lista, il numero di operazioni è direttamente proporzionale alla dimensione della lista.

- **Ricerca Binaria ($O(\log n)$):**

Per una lista ordinata, dividi la lista in due ogni volta. Il numero di operazioni cresce molto lentamente, anche per liste molto lunghe.

Gli hasher nei dizionari

Tutto questo discorso è per dire che i dizionari sono molto efficienti se si vuole trovare un valore, questo perché le chiavi sono hashabili.

Tornando al dizionario di prima:



Python

```
1 m : dict = {"key":1, "key": [1,2,3]}
```

Nel dizionario, **le chiavi devono essere hashabili, mentre i valori possono essere di qualsiasi tipo** :

- **Le chiavi devono essere hashabili** .

Questo è necessario perché i dizionari usano tabelle hash per organizzare e recuperare i valori associati alle chiavi.

- **I ==valori, invece, non devono essere hashabili==**.

Di conseguenza possono essere di qualsiasi tipo, incluso oggetti mutabili come liste o dizionari.

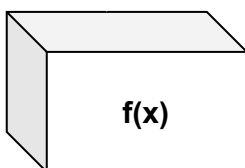
Per valore hashabile si intende: la funzione di hasher ($f(x)$).

Un oggetto è hashabile quando:

- Ha un valore hash immutabile, calcolabile tramite la funzione integrata `hash()` .
- Può essere usato come chiave in un dizionario o elemento di un set.
- È immutabile: tipi come stringhe (`str`), numeri (`int` , `float`) e tuple (se i loro elementi sono hashabili) sono hashabili.

Le funzioni di hash prendono in input, ad esempio una stringa (ciao), e in uscita mi restituiscono un valore numerico (ad esempio: 235) che è associato al valore messo in input.

"Ciao"



Se rimetto il 235 in input non mi esce "ciao" mi restituisce un numero completamente diverso, quindi hashabile significa calcolare il numero in uscita. Detto in altre parole questa proprietà è fondamentale:

l'input uguale produce lo stesso hash value, ma non è garantito il contrario: un hash value non può sempre essere invertito per ricostruire il dato originale.

In Python, alcuni tipi di dati, come stringhe (`str`), numeri (`int` , `float`), e tuple (se tutti i loro elementi sono hashabili), possono essere passati alla funzione `hash()` per ottenere un valore hash (un numero). Tuttavia, tipi mutabili come liste, dizionari o set non possono essere hashabili e non possono essere utilizzati come input per `hash()` .

Tramite le funzioni di hash per le chiavi posso implementare una ricerca più efficiente.



Python

```
1 valore : int =m["CiaoCiao"]
```

Questa espressione, partendo da sinistra: io sto dichiarando una variabile e gli sto dichiarando il tipo `int` , che specifica che la variabile `valore` conterrà un intero (`int`).

Nella parte destra: `m["CiaoCiao"]` accede al valore alla chiave `CiaoCiao` nel dizionario `m` .

Il dizionario usa l'hash della chiave per trovare il valore associato.

Se io ho un dizionario dentro un altro dizionario, come faccio ad accedere al dizionario interno?



Python

```
1 #dichiaro una nuova variabile
2 c:dict ={"i": 1, "j":2, "k" :3}
3 m : dict = {"key":1, "key": [1,2,3]}
4 #sto dichiarando una chiave all'interno di m che ha come valore
  il dizionario c
5 m["inner"]= c
```

Voglio vedere il valore associato a k, come fare?



Python

```
1  #simile alle liste : print(lista_1[4][1])
2  #nei dizionari è simile
3  m ["inner"] ["k"]
```

Sto dicendo a python di andarmi a prendere la chiave `inner` a cui abbiamo associato il dizionario `c`, e mi restituisce il valore associato alla chiave `k`.
Per fare un esempio se ci fosse un'ulteriore chiave



Python

```
1  m["inner"]["k"] ["z"]
```

I dizionari occupano molto spazio in memoria ma rispetto alle liste dove la ricerca è lunga nei dizionari la ricerca è molto più veloce.

Funzione `pop()` :

ti permette di eliminare elementi chiave-valore all'interno del dizionario .

il `.pop()` mi restituisce sempre il valore eliminato

Quindi:



Python

```
1  menu:dict={"menu_estivo": {"Pizza": 5.60, "Pasta": 10,
    "Insalata":5}}
2
3  menu["menu_estivo"]["Pasta"]
4
5  print(menu)
6
7  print(menu["menu_estivo"]["Pasta"])
8
9
10
11  menu_invernale: dict = {"Pizza":20, "Pasta":15, "Insalata": 10}
12
13  menu["menu_invernale"] = menu_invernale
14
15  print(menu)
16
17  # per inserire elementi nuovi
18
19  menu["menu_estivo"]["Bistecca"]=25
```

```

20
21 menu["menu invernale"]["Bistecca"]=30
22
23 # Per eliminare un elemento dal dizionario
24
25 prezzo = menu["menu_estivo"].pop("Pizza")
26
27 print(menu)
28
29 print(f"{prezzo=}")

```

io ho dichiarato una variabile prezzo ho salvato il valore e lo stampato con print, se levo la variabile prezzo la funzione `.pop()` restituisce il valore prezzo ma non lo salva da nessuna parte

Non possono esistere 2 chiavi uguali, perché se io uso la stessa chiave il valore viene sovrascritto :

Voglio far aumentare il valore Pizza nel menu estivo di 150 euro



Python

```

1 menu["menu_estivo"]["Pizza"] = 150

```

Se io adesso voglio modificare il prezzo della pasta nel menu invernale devo fare



Python

```

1 menu["menu invernale"]["Pasta"] = 42

```

Case sensitive delle variabili

Se mi sbaglio scrivendo ad esempio "pasta" in minuscolo mi creerà una nuova chiave con il suo valore

Tuple

Una tupla in Python è una struttura dati immutabile che può contenere una sequenza ordinata di elementi.

Gli elementi di una tupla possono essere di tipi diversi (numeri, stringhe, liste, ecc.), e una volta creata, non può essere modificata (non si possono

aggiungere, rimuovere o modificare elementi).

Le **tuple** sono una collection usata frequentemente con le funzioni, ad esempio per restituire più valori contemporaneamente. I **dizionari** e i **set**, a partire da Python 3.7, preservano l'ordine di inserimento, ma non sono ordinati in senso matematico. Inoltre, i dizionari non permettono chiavi duplicate, ma i loro valori possono essere duplicati.

Le tuple sono ordinate, quindi è possibile fare l'indexing e sono immutabili, cioè non si possono aggiungere o rimuovere elementi ma si possono leggere gli elementi all'interno .

Caratteristiche delle Tuple

- **Immutabilità:**

Una tupla non può essere modificata dopo la creazione.

Questa caratteristica la rende ideale per rappresentare dati che non devono essere alterati.

- **Ordinata:**

Gli elementi di una tupla hanno un ordine specifico, e puoi accedervi tramite indice .

- **Hashabilità:**

- Le tuple stesse sono hashabili e possono essere utilizzate come chiavi in un dizionario o elementi di un set, **a condizione che tutti i loro elementi siano hashabili.**

- **Supporto all'eterogeneità:**

Può contenere elementi di tipi diversi, ad esempio numeri, stringhe e altre tuple.

- **Duplicati:**

le tuple possono contenere dei duplicati

Nel backend le tuple sono molto usate perché essendo immutabili, il loro contenuto non può essere modificato e quindi questo le rende prevedibili e più sicure per rappresentare i dati che non devono cambiare. Inoltre la lettura è più efficiente, proprio a causa della loro immutabilità, e quindi richiedono meno risorse per essere gestite (rispetto alle strutture dati mutabili come le liste).

Quindi Python ne ottimizza la gestione in memoria.

In particolare, la tupla può essere solo letta e di conseguenza l'operazione è più veloce perché non c'è bisogno di controllare se la struttura è stata alterata. Quindi nel backend sono molto utili perché garantiscono efficienza, sicurezza dati e chiarezza semantica.

Sintassi delle Tuple

Per dichiarare una tupla si usa il tipo `:tuple`, e le parentesi tonde `( )`

Per scrivere una tupla:



Python

```
1 my_tuple:tuple =(1,2,3)
```

quindi l'elemento 1 sta all'indice 0, l'elemento 2 all'indice 1 e così via.

≡ Dimostrazione dell'immutabilità delle tuple



Python

```
1 my_tuple:tuple=(1,2,3,4,5,6)
2 my_tuple[0]=10
```

Qua mi darà errore perché le tuple non possono essere modificate.

Alle tuple posso leggere i valori



Python

```
1 my_tuple[0]
```

Mi fa vedere l'elemento all'indice 0.

Se devo dichiarare una tupla con un singolo elemento



Python

```
1 my_tuple :tuple=(8,)
```

perché senno python pensa che voglio leggere l'elemento all'indice zero.

Funzioni delle tuple

funzione. `count` :

mi ritorna il numero di volte che l'elemento è presente nella tupla .

funzione `.index` :

mi ritorna l'indice della prima occorrenza dell'elemento presente nella tupla .

Quando si usano le parentesi

Liste: per dichiarare le liste uso []

i set {} : elenco solo valori

il dizionario {} : dichiaro una chiave : un valore

tuple: dichiarare con le ()

Zucchero sintattico:

quando io uso la not Lista_1[0] io sto usando una funzione, esattamente come l'append, ma è solo sintassi

la sintassi è il modo in cui si scrivono le cose

semantica: cosa quelle cose vogliono dire